

Azure Cosmos DB – Consistency Level Selection Study

Azure Cloud Database Project | B.Tech Implementation Study

Team / Project ID	24CC3046-P048
Domain	Database Azure Cloud
Team Member ID	2400032107
Team Member ID	2400032238
Team Member ID	2400032136
Team Member ID	2400032111

Use cases

- Choose consistency levels for three representative access patterns.
- Balance correctness, latency, availability and Request Unit (RU) consumption.
- Measure the effect of consistency choices using a repeatable test workload.

Bottlenecks students may encounter

- Weaker consistency can introduce stale-read or ordering bugs that are discovered late.
- Strong and bounded-staleness reads can require more RU consumption than weaker levels.
- Test results can vary with region count, workload, item size, indexing and query shape.

1. Abstract

Azure Cosmos DB provides five well-defined consistency levels—Strong, Bounded Staleness, Session, Consistent Prefix and Eventual. This project studies how the choice of consistency level changes application correctness, observed latency, availability characteristics and RU consumption. The team will model three access patterns and compare their behavior under selected consistency configurations. The study is intended as a practical student prototype using controlled sample data rather than production information.

2. Problem Statement

Distributed applications often need to decide whether reads should prioritize the newest committed value, predictable bounded staleness, session-level read-your-writes behavior, ordered updates, or lower consistency constraints. An unsuitable choice can cause stale or unexpectedly ordered reads, while stronger consistency can increase resource consumption and cross-region latency. The project therefore creates a small experimental framework for selecting and documenting an appropriate consistency level for each access pattern.

3. Objectives

- Study all five Azure Cosmos DB consistency levels.
- Define three representative application access patterns.
- Record correctness observations, read/write latency and RU charges.
- Identify cases where stronger consistency is required and cases where weaker consistency is acceptable.
- Produce a decision matrix and architecture diagram for the final demonstration.
- Keep the experiment reproducible with synthetic data and controlled workloads.

4. Proposed Architecture

Azure Cosmos DB - Consistency Level Selection Architecture

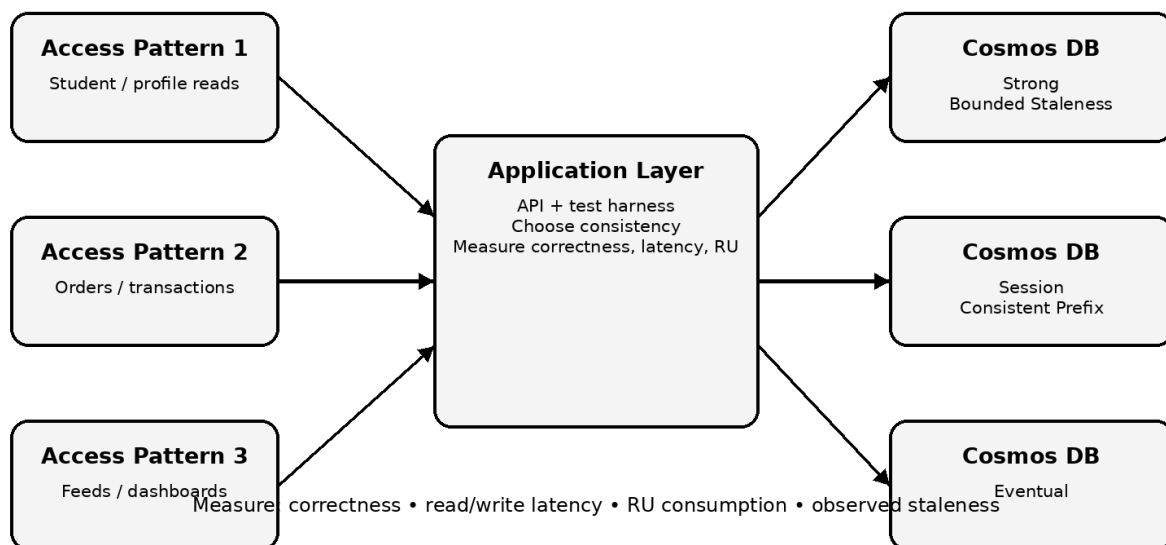


Figure 1. Azure Cosmos DB consistency-level selection test architecture. The supplied .excalidraw file is editable in Excalidraw.

5. Access Patterns and Consistency Study

Access pattern	Example operation	Consistency to test	Main observation
Profile / student reads	Read-after-update profile	Session, Strong	Read-your-writes vs latency/RU
Transactions / orders	Critical status/value read	Strong, Bounded Staleness	Freshness and bounded lag
Feeds / dashboards	High-volume noncritical reads	Eventual, Session, Consistent Prefix	Latency/RU vs stale or ordered reads

Azure Cosmos DB documents five levels from strongest to weakest: Strong, Bounded Staleness, Session, Consistent Prefix and Eventual. Strong provides linearizable reads; Bounded Staleness limits lag by configured versions/time; Session provides read-your-writes behavior within a client session; Consistent Prefix preserves ordering guarantees; Eventual allows the greatest staleness flexibility. (Microsoft Learn)

6. Azure Services / Technologies

Service / Technology	Purpose	Project Use
Azure Cosmos DB for NoSQL	Distributed database	Store synthetic documents and run consistency experiments
Azure Portal	Cloud management and monitoring	Configure consistency and inspect metrics
Cosmos DB SDK / REST API	Application access	Run repeatable read/write workloads
Azure Monitor / Metrics	Observability	Capture latency, throughput and consistency-related measurements
Excalidraw	Architecture visualization	Create/edit the project architecture diagram

RU charges are the project's cost metric. Microsoft documents that strong and bounded-staleness reads use quorum reads and can consume about twice the read RUs of the other consistency levels for the same workload. Actual RU consumption still depends on item size, indexing, query shape and result size.

7. Implementation Plan

1. Create a small synthetic dataset representing profiles, transactions and feed/dashboard records.
2. Create an Azure Cosmos DB account/container and document the selected default consistency level.
3. Implement a simple test harness that performs equivalent reads and writes under the chosen consistency configurations.
4. Capture response latency and RU charge for each operation.
5. Test read-after-write behavior and ordering scenarios where applicable.
6. Repeat each workload enough times to reduce the effect of one-off measurements.
7. Compare results in a decision matrix and document the trade-offs.
8. Capture Azure Portal evidence and finalize the Excalidraw architecture diagram.

8. Bottlenecks and Mitigation

Bottleneck	Mitigation
Weaker consistency introduces bugs found late	Create explicit stale-read and ordering test cases before performance testing.
Request Unit provisioning drives unexpected cost	Start with small synthetic workloads, record RU charges per operation, and avoid unnecessary high-
Latency measurements vary between runs	Use repeated runs, fixed item sizes and stable query patterns; report observations rather than a sing
Global-region behavior is difficult to reproduce	Begin with a controlled single-region experiment and add multi-region tests only if resources allow.
Configuration mistakes affect results	Record account/container settings and test one variable at a time.

9. Study Acceptance Criteria

- Five consistency levels are documented with their guarantees and intended test scenarios.
- Three access patterns are implemented with synthetic sample data.
- Each tested workload records correctness observations, latency and RU consumption.
- At least one stale-read or ordering behavior is demonstrated where the selected consistency model permits it.
- A comparison table explains the trade-offs without treating one level as universally optimal.
- Architecture diagram is available as an editable Excalidraw file and as a report figure.

10. Team Demo Pitch

Our project studies consistency as an application-design decision rather than simply a database setting. We model three access patterns, run controlled experiments on Azure Cosmos DB, and measure correctness, latency and RU consumption. The final output shows how different consistency guarantees affect application behavior and provides a documented basis for selecting a level for a specific workload.

11. Suggested GitHub Structure

```
azure-cosmos-consistency-study/  
■■■ README.md  
■■■ docs/ ABSTRACT.md · PROBLEM.md · CONSISTENCY.md · IMPLEMENTATION.md · BOTTLENECKS.md · DEMO-SCRIPT.md  
■■■ architecture/ cosmos-consistency-architecture.excalidraw  
■■■ src/ test_harness.py · workloads.py  
■■■ sample-data/ profiles.json · transactions.json · feeds.json  
■■■ evidence/ screenshots/ · results.csv
```

12. Conclusion

The project provides a practical way to study the trade-off between consistency guarantees and system performance in Azure Cosmos DB. The experiment focuses on measurable behavior—correctness, latency, RU consumption and observed staleness—so the team can document a consistency choice for each access pattern rather than using one setting for every workload.

References: Microsoft Learn – Consistency level choices; Request Units; Manage Consistency.